The Automatic Statistician, revisited

A technical report on the repository

Warith Harchaoui

9 November 2026

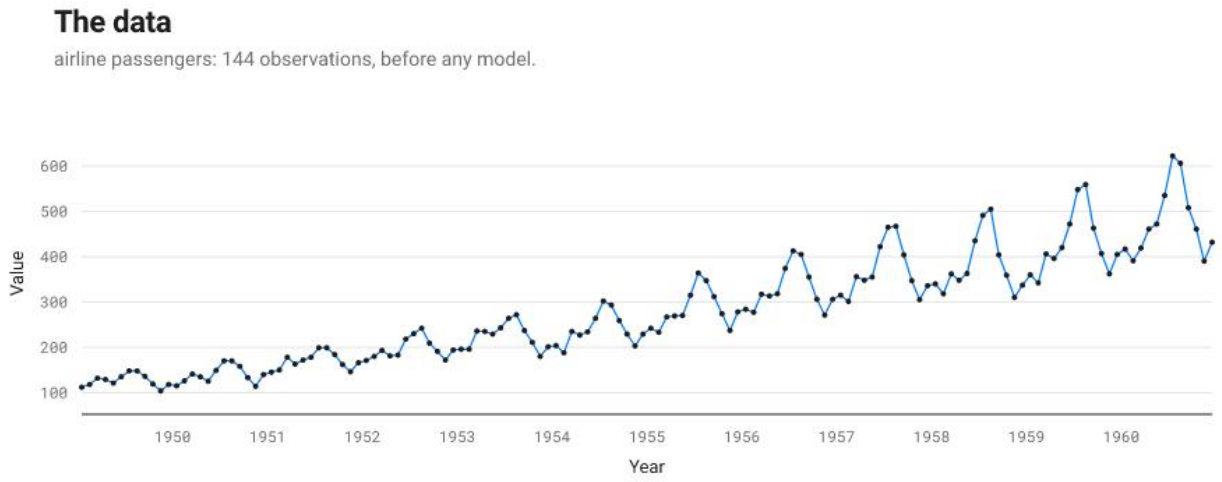


Figure 1: Monthly airline passengers from 1949 to 1960, in thousands. This series runs through the whole report, as it ran through the 2014 paper.

Summary

In 2014 a program read twelve years of airline passengers and wrote, in plain English, that the series holds a linear trend, a yearly cycle whose amplitude grows with the trend and noise that grows too. Nobody told it to look for a year. That program was the Automatic Statistician [14, 37], written in Python 2.7 and MATLAB.

This repository brings it back to life in Python 3 with `torch`. It also ports AutoGP [47] from Julia and runs the 2014 extrapolation benchmark again, this time against the forecasting methods of 2026.

The program works in four steps:

1. Given a time series, the structure search looks for the expression over Gaussian-process kernels that explains the data best by the Bayesian information criterion (BIC) [49].
2. The search splits that expression into additive components and describes each one in English.
3. Each component is checked against its prior [36].
4. A report gathers everything. It now writes the expression as a formula with its fitted parameters. It also says how much the data prefer the kept structure to the next one.

The main results:

- **The port is faithful.** The 2014 system published 130 models (13 series, 10 search levels each). All 130 reproduce their negative log likelihood: twelve series to within 5×10^{-5} nats, the ill-conditioned solar series to within 5×10^{-3}. The component sentences of the airline report match the original word for word. Two series have been searched again from scratch so far. On both, the ten-level search reaches a BIC at least as good as the published one (lower is better): airline 1108.9 against 1123.7, solar -872.3 against -870.3.
- **AutoGP agrees with its original.** Every deterministic quantity of the port matches AutoGP.jl to 10^{-12} or better. The comparison also turned up a surprise: the Hamiltonian moves of AutoGP.jl see only the gradient of the prior.
- **The 2014 forecasts hold up in 2026.** On the thirteen series, 24 methods are compared by mean standardised error. The published ABCD forecasts come fifth by mean (2.05), behind three pre-trained foundation models and N-BEATS [3, 39, 2, 40]. They never fail badly: the worst series scores 4.00. Eureqa [48], the symbolic-regression baseline of 2014, has a comparable median (2.82). Its mean climbs to 48.11 because of one series, where its formula extrapolates 590 times worse than the best method. PySR [11], run here on the same split, stays bounded on that series (4.56) and has a mean of 2.76.
- **The structure search is a symbolic regression.** Its grammar builds kernel expressions where symbolic regression builds arithmetic ones. The reports now state the formula found. The section "What the formula contains" shows what that formula holds and what it leaves out. Eureqa, PySR and their successors read more easily. The structure search models noise and uncertainty. It also extrapolates somewhat better than PySR here. On smooth components, their formulas and ours complement each other.

The system

What the search looks for

A Gaussian process (GP) [44] is a probability distribution over functions. With a zero mean, as here, the covariance $k(t, t')$ fixes it. It says how strongly the values at times t and t' move together. The search builds covariances from base kernels:

symbol	base kernel	what it describes
`SE`	squared exponential	a smooth function
`Per`	periodic	a repeating pattern, with its period
`Lin`	linear	a linear trend (products of them make polynomials)
`C`	constant	an offset
`WN`	white noise	uncorrelated noise
`RQ`	rational quadratic (2013 grammar only)	smoothness at several scales

Operators combine them. A sum adds independent components. A product modulates one component by another: `Lin x Per` is a cycle whose amplitude grows linearly. Two operators of the 2014 grammar switch between regimes: the changepoint `CP` and the change window `CW`.

One level of the search runs as follows:

1. The grammar rewrites one subexpression of each current expression at a time. The rewrites are the candidates.
2. Each candidate's parameters are fitted by maximising the GP marginal likelihood (L-BFGS here, conjugate gradients in GPML [43]).
3. The candidates are ranked by BIC. The best few seed the next level.

The 2014 configuration searches ten levels.

From expression to report

The winning kernel is rewritten as a sum of products. Each product is one additive component. The components are ordered by how much each one lowers the cross-validated mean absolute error of extrapolation. The original's rule system then describes each one: "An approximately periodic function with a period of 1.0 years and with linearly increasing amplitude."

Model checking compares each component's posterior with its prior through four statistics: quantiles, autocorrelation, periodogram and maximum mean discrepancy [19, 36]. The report comes in two formats, as the 2014 system wrote it. md2star compiles the Markdown version to PDF and `latexmk` compiles the LaTeX version. Sprezzature-figures draws the figures.

AutoGP: a posterior over structures

AutoGP [47] grew out of Bayesian synthesis of probabilistic programs [46]. It replaces the greedy search by sequential Monte Carlo. The data enter in batches. A population of structures is reweighted by how well each one predicted the batch, then resampled, then moved by structure proposals and by Hamiltonian Monte Carlo on the parameters. The result is a weighted posterior over structures. Their forecasts are averaged. `autostat.smc` ports AutoGP.jl [45]. Each heavy structure of its posterior gets the 2014 report and its checks.

Surfaces

The same service layer answers from six places:

- a Python library
- a command line written with Click
- a local GUI generated by sprezzature-cli-gui
- an HTTP API written with FastAPI
- an MCP server (FastAPI-MCP, four tools)
- an Agent Skill, routed by `TRIGGERS.md`

How faithful the port is

The port keeps what decides the results: the original's parameterisation, grammar, simplification rules, initialisation heuristics, BIC parameter count and greedy loop (`PORTING.md`). It changes three things. The optimiser is L-BFGS. The covariances are written in `torch`, where autograd replaces hand-written derivatives. The parallelism is new. `PORTING.md` also documents four defects found in the original. None of them changes a published number.

check	result
NLL of the 130 published models	12 series within 5×10^{-5} nats, solar within 5×10^{-3}
airline component sentences	identical to 2014
gradients against finite differences	within 10^{-4}
ten-level search, final BIC (lower is better), two series so far	airline 1108.9 vs 1123.7, solar -872.3 vs -870.3

For AutoGP, `tests/test_smc_julia.py` loads AutoGP.jl v0.1.19 through `juliacall` and compares the two implementations number by number:

- the joint densities of 30 structures agree to 10^{-14}
- the covariance matrices agree to 10^{-15}
- the densities of 400 structure moves agree to 10^{-15} and their density ratios to 4×10^{-13}
- the predictive moments agree to 3×10^{-12}

The surprise came from the gradients. On all 359 kernel parameters tested, the gradient Gen computes equals the gradient of the parameter's standard normal prior. The reason is that `covariance_prior` is not declared differentiable, so the likelihood never reaches the leapfrog steps. The Hamiltonian moves of AutoGP.jl explore the prior, where the intended sampler explores the posterior. The port reproduces this behaviour by default and offers the intended sampler as `HMCConfig(gradient="full")`.

Extrapolation against the methods of 2026

The 2014 benchmark trains on the first 90 % of thirteen series and forecasts the last 10 %. The repository first recomputes the paper's table from the published forecasts, to within 2.2×10^{-10}. It then scores these methods of 2026 on the same split:

- classical models selected by information criterion [25, 24, 15]
- LightGBM [29] and deep networks [40, 10], tuned by rolling-origin cross-validation on the training part only
- pre-trained foundation models, used zero-shot [4, 3, 12, 18, 39, 42].

The score is the standardised root mean squared error (RMSE): each series' error divided by the best error on that series. A score of 1.00 means the method was the best on that series. A score of 2.00 means its error was twice the best one.

method	family	median	mean	series won
Chronos-2	foundation	1.47	1.65	1
Chronos-Bolt (base)	foundation	1.53	1.70	1
Timestral 209M	foundation	1.57	1.65	1
KumoTabular on lags	foundation	1.68	2.50	2
TimesFM 2.5	foundation	1.72	2.17	0
AutoARIMA	classical	1.79	3.22	0
N-HiTS	deep learning	1.81	2.09	1
AutoTheta	classical	1.89	2.21	0
N-BEATS	deep learning	1.91	1.96	0
AutoETS	classical	1.96	2.45	0
LightGBM on lags	machine learning	2.08	4.38	1
ABCD 2014	structure search	**2.12**	**2.05**	1
Spectral mixture [59]	2014 baseline	2.19	3.22	0
CKS 2013	structure search	**2.48**	**2.17**	2
Eureqa [48]	symbolic regression (2014 published)	**2.82**	**48.11**	1
PySR [11]	symbolic regression	**2.87**	**2.76**	0
gplearn [53]	symbolic regression	3.60	8.84	0

The table is abridged from `benchmark/RESULTS.md`, which lists all 24 methods. Two readings matter.

First, foundation models, trained on large corpora, now extrapolate these series best. Chronos-2 has a median standardised error of 1.47 and ABCD 2.12.

Second, among the methods that fit one model to the series at hand, the structure search is the most reliable. Its mean (2.05) sits below its median (2.12), so no series goes badly. Eureqa's mean comes from a single series, monthly average daily calls, where its formula scores 590.55 against 1.20 for ABCD. Without that series, Eureqa's mean falls to 2.90.

The structure search as symbolic regression

The parallel

Symbolic regression [32] searches a grammar of expressions, fits each expression's constants and selects by trading accuracy against complexity. The structure search does the same with another grammar. Its expressions denote covariances where symbolic regression's denote regression functions. Marginal likelihood fits its constants. BIC plays the role of the complexity penalty. Both problems are NP-hard [58], so both communities search heuristically.

method	search space	search	selection	output	noise
Eureqa [48]	arithmetic trees	genetic programming	accuracy-parsimony front	closed form	implicit
PySR [11]	arithmetic trees, user operators	evolution with annealing	loss-complexity front	closed forms	implicit, custom loss
AI Feynman [56, 57]	arithmetic, recursively simplified	neural-guided decomposition	description length	closed form	robust (version 2.0)
SINDy [8]	span of a fixed library	thresholded least squares	sparsity	closed-form ODE	smoothing
DSR [41]	token sequences	policy gradient	reward	closed form	implicit
Transformers [27, 5]	pre-training distribution	amortised decoding	fit	closed form	via training data
LLM-SR [50]	Python programs	proposals from a language model	fit	program	implicit
Bayesian machine scientist [21]	arithmetic trees	MCMC	posterior, BIC likelihood	closed forms	Gaussian
CKS 2013 [14]	kernel grammar, + and ×	greedy beam	BIC	kernel, GP posterior	Gaussian
ABCD 2014 [37]	kernel grammar + CP, CW, WN	greedy beam	BIC	kernel, report, **formula**	explicit, heteroscedastic
AutoGP [47]	kernel grammar	SMC	posterior weight	weighted kernels	Gaussian

Symbolic-regression benchmarks [33, 17, 1, 51] measure accuracy and the recovery of known equations on regression tasks. None of them holds an extrapolation task on real time series, the very task where the two families differ most.

What the formula contains

Each report now has a section called "The formula found". For the airline series, the published 2014 model reads

$$y(t) = f_1(t) + f_2(t) + f_3(t) + f_4(t)$$

$$f_1(t) \sim \mathrm{GP}\left(0,\ 1122\,(t-1946)(t'-1946)\right), \qquad f_1(t) = 92.68 + 31.58\,(t-1949)$$

$$f_2(t) \sim \mathrm{GP}\Big(0,\ 21.91\,(t-1945)(t'-1945) \cdot \frac{\exp\left(1.924\cos\frac{2\pi(t-t')}{1.002}\right) - I_0(1.924)}{e^{1.924} - I_0(1.924)} \cdot \exp\left(-\frac{(t-t')^2}{2\cdot 22.98^2}\right)\Big)$$

$$f_3(t) \sim \mathrm{GP}\left(0,\ 0.285\,(t-1943)(t'-1943)\cdot\delta_{tt'}\right), \qquad f_4(t) \sim \mathrm{GP}\left(0,\ 193.1\,\exp\left(-\frac{(t-t')^2}{2\cdot 0.6787^2}\right)\right)$$

Each line answers a question a statistician would ask:

- The trend f_1 has a closed form, read exactly from its posterior mean: 31.6 thousand more passengers each year.
- The cycle f_2 has a period of 1.002 years, with an amplitude that grows linearly from 1945. The data found the calendar year to within two thousandths of a year, less than a day.
- The noise f_3 grows linearly too.
- The term f_4 is a smooth function with a length scale of 0.68 years.

The solar series gives a more striking result. Its formula shows a change window $w_1(t)$ from 1643 to 1716, inside which a constant $f(t) = -0.5964\,w_1(t)$ lowers solar irradiance by 0.6 W/m². These are the years of the Maunder minimum, when sunspots almost vanished. Nobody told the search about them. It found them!

Every component built from constants and linear kernels gets a closed form like these, even when a change of regime switches it on or off. Periodic components get one too, because the periodic kernel is a sum of cosines:

$$\exp(a\cos u) = I_0(a) + 2\sum_{k\geq 1} I_k(a)\cos(ku).$$

A periodic component's posterior mean is therefore a Fourier series in the period found. Its weights fall quickly when a is small. The report writes the series up to the harmonic after which the rest weighs less than 0.1 % of the component and marks it with $\approx$.

On the thirteen published models this gives a closed form to 16 of the 76 components: trends and offsets, the yearly cycle of the temperature series (amplitude 4.3 degrees), the weekly and half-yearly cycles of the births series. Three kinds of component have none: smooth components, periodic ones modulated by a smooth kernel and noise. A draw from such a process is no finite formula, so its covariance serves as its formula. Here the structure search parts ways with arithmetic symbolic regression. Its expression reads less explicitly. In exchange, the expression includes its own uncertainty, which keeps it from extrapolating the way an unconstrained formula does.

Recovering a known formula

Examples 4 and 5 use synthetic monthly sales from 2016 to 2023, drawn from

$$y = 100 + 0.5\,m + 8\sin(2\pi m/12) + \varepsilon,$$

with m the month index and $\varepsilon \sim \mathrm{N}(0, 2^2)$. After five levels the search keeps `Lin + WN + Per`. A sixth level does not improve on it. Here is the formula it reports, with t in years:

$$f_1(t) = 99.63 + 6.095\,(t-2016)$$

$$f_3(t) \approx 7.95\cos\left(2\pi(t-2016) - 1.807\right) + 0.3961\cos\left(2\pi\cdot 2\,(t-2016) - 0.9546\right)$$

$$f_2(t) \sim \mathrm{GP}\,(0,\ 3.578\,\delta_{tt'})$$

The truth has a slope of 0.5 per month, so 6 per year, starting from 100. A sine of amplitude 8 repeats every year. The noise variance is 4. The search finds a slope of 6.095, an amplitude of 7.95 and a noise variance of 3.578. It also adds two things of its own: a second harmonic (5 % of the first) and a phase offset of 0.24 radians (about two weeks). Part of both comes from the way dates become decimal years with 365-day years, which do not sample the months at equal steps.

Where autostat stands among symbolic-regression tools

Symbolic regression returns a formula $y = f(t)$. Autostat, the package of this repository, returns an expression over covariances, $y = f_1(t) + \cdots + f_n(t)$ with each f_i a Gaussian process. It writes in closed form the components that have one. The two tools answer different questions. The benchmark shows where the difference matters, on four counts.

Extrapolation. The benchmark holds three symbolic-regression systems. Each fits $y = f(t)$ on the first 90 % and continues the formula. All three fail worst on the same series, monthly average daily calls, by amounts two orders of magnitude apart: 590.55 for Eureqa, 74.08 for gplearn, 4.56 for PySR. The blow-up of 2014 therefore belongs to Eureqa's formula, because PySR, a free formula too, stays bounded. PySR's mean (2.76) and median (2.87) stay behind those of ABCD (2.05 and 2.12, worst series 4.00). The margin is modest and comes from one seeded run of 60 iterations per series. Operon [9] was tried and did not run, because its macOS wheel does not load.

Noise. Most symbolic-regression systems minimise squared error, so noise is whatever is left over [11, 33]. The grammar searched here models noise as a component of its own: white noise, noise growing with time (`Lin x WN`), noise confined to a period by a change of regime. The formula of the sales example reports its noise variance (3.578 against a true 4) the way it reports its slope.

Selection. Symbolic regression selects on an accuracy-complexity front, usually without a probabilistic score. The structure search selects by BIC, an approximation of the marginal likelihood. Its closest arithmetic counterpart is the Bayesian machine scientist [21]. It also scores expressions with a likelihood approximated by BIC and samples them by Markov chain Monte Carlo, as AutoGP does for kernels.

Readability. Symbolic regression wins this one. A smooth component has no closed form in this repository, only its covariance and a sentence ("A smooth function"). Of the 76 components of the thirteen published models, 16 have an exact or Fourier closed form.

Symbolic regression also helps the structure search. Each component's posterior mean is a denoised curve in one dimension, the easiest input a symbolic regressor ever gets. On request (`symbolic=True`), PySR reads each smooth component. Its shortest formula is shown, marked ≈, only if it reproduces the component to within 1 %. The covariance remains the model. The formula is a reading of it.

On the thirteen published models, PySR tried 13 smooth components and found a formula within 1 % for 4 of them, with largest gaps from 0.083 to 0.72 %. The other 9 kept their covariance. Twenty of the 76 components then read as a formula (16 and 4). The 9 left include fast-varying curves such as the airline's smooth term, whose length scale is 0.68 years.

Two other families suit this task less well. SINDy [8], built for dynamical systems, expects the derivatives of a state. LLM-SR [50] calls a large language model for each expression.

Examples and search depth

`examples/` holds one script per example of `EXAMPLES.md`, with the reports each script writes in both formats. The original configuration has no stopping rule: it always searches `max_depth` levels. The first examples stopped at a small fixed depth. Searching deeper found more:

example	depth used	BIC	deeper search	BIC
airline, 2014	2	1236.4 (3 components)	11 levels with patience 2, converged at level 9	1108.9 (4 components)
airline, 2013	3	1370.4 (2 components, no cycle)	10 levels, 13 minutes	1112.4 (yearly cycle found from level 4)
sales, 2014	3	458.4 (`WN + Lin x SE + Per x SE`)	converged at level 5	449.2 (`Lin + WN + Per`, the generating formula)

The sales series of examples 4 and 5 is synthetic: a linear trend, a yearly sine and Gaussian noise. At three levels the search approximates it with smooth terms. At five it finds `Lin + WN + Per`, the very formula the data were drawn from. A sixth level does not improve on it. The examples now search until BIC stops falling, then two levels further, up to twenty (patience 2, described below). The published 2014 models of examples 2 and 8 come from the full ten-level search and need no extension.

The stopping criterion

The 2014 configuration always searches ten levels and keeps the lowest BIC. The thirteen published searches show what that choice buys. For each series, the table below gives five things:

- the level kept
- the level where its structure first appeared (the levels in between only refit its parameters)
- the gap to the best different structure, read on the Kass and Raftery scale [28]
- what a rule stopping after one level without a BIC fall above 0.1 would have done (patience 1), then the same rule with two levels (patience 2)
- the number of components, with the number of closed forms in brackets

series	kept level	structure first reached	best rival (ΔBIC)	patience 1	patience 2	components (closed forms)
01-airline	10	7	6.2 (strong)	stops at 10, same model	does not stop	4 (1)
02-solar	10	9	4.5 (positive)	does not stop	does not stop	8 (2)
03-mauna	10	7	38.5 (very strong)	does not stop	does not stop	5 (0)
04-wheat	10	5	6.0 (strong)	stops at 8, same model	stops at 9, same model	4 (1)
05-temperature	9	6	5.2 (positive)	stops at 8, same model	stops at 9, same model	4 (2)
06-internet	10	10	26.3 (very strong)	stops at 7, worse model (+29.2)	stops at 8, worse model (+29.2)	7 (1)
07-call-centre	10	10	1.9 (barely worth mentioning)	does not stop	does not stop	6 (2)
08-radio	8	5	3.5 (positive)	stops at 7, same model	stops at 8, same model	6 (2)
09-gas-production	10	10	4.6 (positive)	does not stop	does not stop	7 (0)

series	kept level	structure first reached	best rival (ΔBIC)	patience 1	patience 2	components (closed forms)
10-sulphuric	10	8	18.7 (very strong)	does not stop	does not stop	9 (1)
11-unemployment	9	5	66.3 (very strong)	stops at 10, same model	does not stop	6 (1)
12-births	5	5	2.3 (positive)	stops at 6, same model	stops at 7, same model	4 (2)
13-wages	10	8	11.4 (very strong)	stops at 7, worse model (+11.4)	does not stop	6 (1)

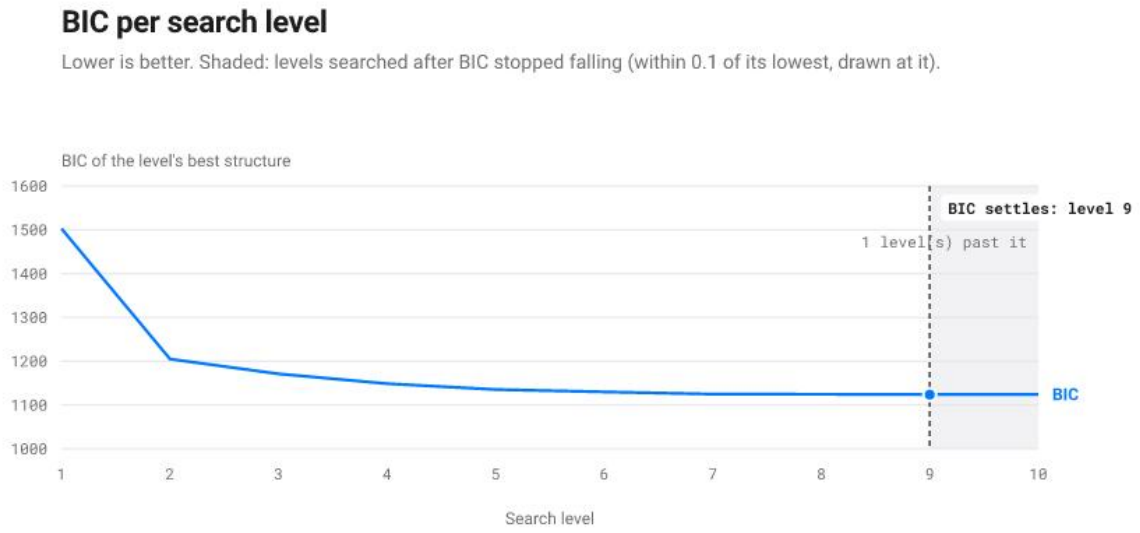


Figure 2: BIC per level of the published airline search. The structure settles at level 7. The last three levels only refit it.

The table supports four findings.

- **Ten levels are sometimes too few.** On internet, call-centre and gas-production, the kept structure appears at level 10, the last one, so the search was still changing the expression when it was cut. On solar it appears at level 9.
- **A flat BIC is no proof of convergence.** Stopping at the first level without improvement would have kept a worse model on two series: internet by 29.2 (very strong evidence against it) and wages by 11.4. Two levels of patience repair wages but not internet, where the search stalls for two levels before finding a much better structure.
- **Much of the depth goes into refitting.** On nine series the kept structure is found by level 8. The later levels only refit its parameters. On six of them that refit is worth 5.5 BIC or less. It is worth 9.3 on sulphuric, 35.5 on mauna and 44.2 on unemployment. On those three, the random restarts find better parameters for an expression the search already has. Depth buys optimisation as well as structure.
- **The last level is not always the best.** On temperature, radio, unemployment and births, the kept model comes from an earlier level. The later levels, restarted from random initialisations, fitted no better.

The evidence for the kept structure against its best rival is very strong on five series, strong on two and

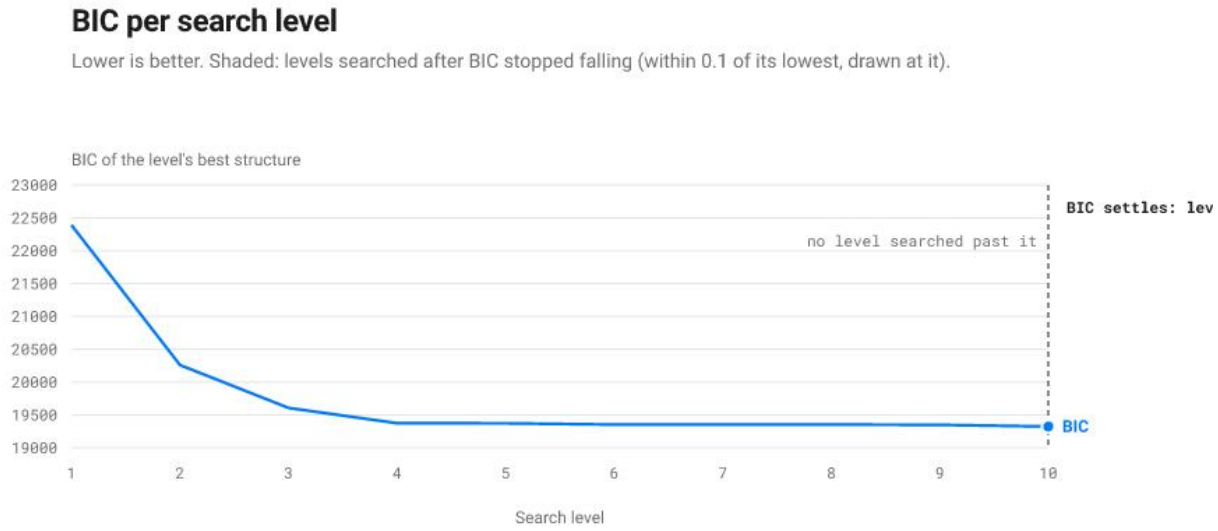


Figure 3: BIC per level of the published internet search. BIC stays flat for two levels, then a structure better by 29 BIC appears at level 10.

positive on five. On call-centre it is barely worth mentioning (1.9): two expressions explain the data about equally well. Read that report as one of the two.

The examples in `examples/` therefore search until BIC has not fallen by 0.1 for two levels in a row, with at most twenty levels, then report the levels past the best one. Each report has a section "Search path and stopping criterion" with the BIC curve, the table above for its own search and these readings. When the search ran a single level, the report says there is no curve to draw.

How much the data prefer the kept structure

A gap in BIC is hard to feel. How much more likely is the kept structure than its rival when the gap is 1.9? Or 4.6?

Half the BIC approximates minus the log of the marginal likelihood. So $\exp(-\mathrm{BIC}/2)$ is, up to a constant shared by all structures, how likely each structure makes the data. Give every structure the same prior weight and normalise over the structures the search met, each with its lowest BIC. Each structure then has a posterior weight. The Bayesian machine scientist [21] weighs its arithmetic expressions the same way.

The order of magnitude checks out by hand. A gap of 4.6 gives a ratio of $\exp(-2.3) \approx 0.10$, so the rival is ten times less likely than the kept structure. The sales example of `EXAMPLES.md` shows exactly that, with weights of 0.899 for `Lin + WN + Per` and 0.092 for `Lin + WN + Per x SE`. On the published call-centre search, the gap of 1.9 gives 0.713 to the kept structure and 0.281 to the next.

Each report now states these two weights in one sentence and reads them on three levels, according to the rival's weight:

- above 0.25, the data barely tell the two structures apart (call-centre)
- between 0.05 and 0.25, the data favour the kept one without ruling the next out (sales)
- below 0.05, the kept structure takes almost all the weight

The section "Search path and stopping criterion" adds the table of weights for every structure met with a weight of at least 0.001.

AutoGP needs a different picture, because it has no search levels and so no BIC curve. Its summary report draws one horizontal stem per structure kept by the posterior. The stem is as long as that structure's BIC

above the lowest and carries the structure's posterior weight as a label. A sentence then says whether BIC agrees with the posterior on which structure comes first. With a single structure, the summary says there is no BIC to compare.

Limits

- **Cost.** Exact GP inference grows with the cube of the number of points, so the series are binned to 1,000 points. The full airline search took 4.1 hours on six processes. Scalable bounds [30] and Bayesian optimisation over the grammar [38, 6] search with fewer evaluations. Neither is ported.
- **One input.** The search handles one time input. Multivariate grammars [13] and structure shared across series [23] are not ported.
- **Formulas.** Without PySR, or when no short formula comes within 1 %, smooth components keep their covariance as their formula.
- **Benchmark.** TabPFN-TS [22] did not run because of its licence. AutoGP.jl, GPy-ABCD [16], AutoBNN [31] and Moirai [60] did not install on the benchmark machine.

Related work not compared

Neural kernel networks learn kernel sums and products by gradient descent [54]. Spectral mixtures fix a flexible form [59]. Language models now propose kernels [55], propose probabilistic programs [35] or judge fits from plots [34]. Gallifrey reimplements AutoGP in JAX for astronomy [7]. Steinruecken et al. [52] summarise the Automatic Statistician project. The project's grammar descends from the matrix decomposition grammar of Grosse et al. [20]. Bayesian symbolic regression [26, 21] is the closest arithmetic counterpart to a posterior over structures.

References

[1] Guilherme S. Imai Aldeia et al. *Call for Action: towards the next generation of symbolic regression benchmark*. GECCO '25 Symbolic Regression Workshop; proposes SRBench 2.0. 2025. arXiv: 2505.03977. URL: https://arxiv.org/abs/2505.03977.

[2] Amazon Web Services. *Chronos-Bolt (base): model card*. Hugging Face model amazon/chronos-bolt-base, published 25 November 2024, Apache 2.0; no separate paper. 2024. URL: https://huggingface.co/amazon/chronos-bolt-base.

[3] Abdul Fatir Ansari et al. *Chronos-2: From Univariate to Universal Forecasting*. 2025. arXiv: 2510.15821. URL: https://arxiv.org/abs/2510.15821.

[4] Abdul Fatir Ansari et al. "Chronos: Learning the Language of Time Series". In: *Transactions on Machine Learning Research* (2024). arXiv: 2403.07815. URL: https://openreview.net/forum?id=gerNCVqqtR.

[5] Luca Biggio et al. "Neural Symbolic Regression that Scales". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021, pp. 936–945. arXiv: 2106.06427. URL: https://proceedings.mlr.press/v139/biggio21a.html.

[6] Matthias Bitzer, Mona Meister, and Christoph Zimmer. "Structural Kernel Search via Bayesian Optimization and Symbolical Optimal Transport". In: *Advances in Neural Information Processing Systems 35*. 2022. arXiv: 2210.11836. URL: https://papers.neurips.cc/paper_files/paper/2022/hash/ff7373914a96956f2a7cacbdf3b0b8d8-Abstract-Conference.html.

[7] Christopher Boettner. "gallifrey: JAX-based Gaussian process structure learning for astronomical time series". In: *Astronomy & Astrophysics* 699 (2025), A42. DOI: 10.1051/0004-6361/202554518. arXiv: 2505.20394.

[8] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. "Discovering governing equations from data by sparse identification of nonlinear dynamical systems". In: *Proceedings of the National Academy of Sciences* 113.15 (2016), pp. 3932–3937. DOI: 10.1073/pnas.1517384113. arXiv: 1509.03580.

[9] Bogdan Burlacu, Gabriel Kronberger, and Michael Kommenda. "Operon C++: An Efficient Genetic Programming Framework for Symbolic Regression". In: *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion (GECCO '20)*. ACM, 2020, pp. 1562–1570. DOI: 10.1145/3377929.3398099.

[10] Cristian Challu et al. "NHITS: Neural Hierarchical Interpolation for Time Series Forecasting". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 37. 6. 2023, pp. 6989–6997. DOI: 10.1609/aaai.v37i6.25854. arXiv: 2201.12886.

[11] Miles Cranmer. *Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl*. Software: https://github.com/MilesCranmer/PySR. 2023. arXiv: 2305.01582. URL: https://arxiv.org/abs/2305.01582.

[12] Abhimanyu Das et al. "A decoder-only foundation model for time-series forecasting". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 10148–10167. arXiv: 2310.10688. URL: https://proceedings.mlr.press/v235/das24c.html.

[13] David Duvenaud, Hannes Nickisch, and Carl Edward Rasmussen. "Additive Gaussian Processes". In: *Advances in Neural Information Processing Systems 24*. 2011. arXiv: 1112.4394. URL: https://arxiv.org/abs/1112.4394.

[14] David Duvenaud et al. "Structure Discovery in Nonparametric Regression through Compositional Kernel Search". In: *Proceedings of the 30th International Conference on Machine Learning*. Vol. 28. Proceedings of Machine Learning Research 3. PMLR, 2013, pp. 1166–1174. arXiv: 1302.4922. URL: https://proceedings.mlr.press/v28/duvenaud13.html.

[15] Jose A. Fiorucci et al. "Models for optimising the theta method and their relationship to state space models". In: *International Journal of Forecasting* 32.4 (2016), pp. 1151–1161. DOI: 10.1016/j.ijforecast.2016.02.005.

[16] Thomas Fletcher, Alan Bundy, and Kwabena Nuamah. "GPy-ABCD: A Configurable Automatic Bayesian Covariance Discovery Implementation". In: *8th ICML Workshop on Automated Machine Learning (AutoML 2021)*. 2021. URL: https://www.research.ed.ac.uk/en/publications/gpy-abcd-a-configurable-automatic-bayesian-covariance-discovery-i/.

[17] Fabrício Olivetti de França et al. "SRBench++: Principled Benchmarking of Symbolic Regression With Domain-Expert Interpretation". In: *IEEE Transactions on Evolutionary Computation* 29.4 (2025), pp. 1127–1137. DOI: 10.1109/TEVC.2024.3423681.

[18] Google Research. *TimesFM 2.5 (200M, PyTorch): model card and code*. Hugging Face model google/timesfm-2.5-200m-pytorch, published September 2025, Apache 2.0; code at https://github.com/google-research/timesfm. 2025. URL: https://huggingface.co/google/timesfm-2.5-200m-pytorch.

[19] Arthur Gretton et al. "A Kernel Two-Sample Test". In: *Journal of Machine Learning Research* 13.25 (2012), pp. 723–773. URL: https://jmlr.org/papers/v13/gretton12a.html.

[20] Roger Grosse et al. "Exploiting Compositionality to Explore a Large Space of Model Structures". In: *Proceedings of the Twenty-Eighth Conference on Uncertainty in Artificial Intelligence (UAI)*. 2012. arXiv: 1210.4856. URL: https://arxiv.org/abs/1210.4856.

[21] Roger Guimerà et al. "A Bayesian machine scientist to aid in the solution of challenging scientific problems". In: *Science Advances* 6.5 (2020), eaav6971. DOI: 10.1126/sciadv.aav6971.

[22] Shi Bin Hoo et al. *From Tables to Time: Extending TabPFN-v2 to Time Series Forecasting*. Software: https://github.com/PriorLabs/tabpfn-time-series. 2025. arXiv: 2501.02945. URL: https://arxiv.org/abs/2501.02945.

[23] Yunseong Hwang, Anh Tong, and Jaesik Choi. "Automatic Construction of Nonparametric Relational Regression Models for Multiple Time Series". In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. arXiv title: The Automatic Statistician: A Relational Perspective. PMLR, 2016, pp. 3030–3039. arXiv: 1511.08343. URL: https://proceedings.mlr.press/v48/hwangb16.html.

[24] Rob J. Hyndman and Yeasmin Khandakar. "Automatic Time Series Forecasting: The forecast Package for R". In: *Journal of Statistical Software* 27.3 (2008). DOI: 10.18637/jss.v027.i03.

[25] Rob J. Hyndman et al. "A state space framework for automatic forecasting using exponential smoothing methods". In: *International Journal of Forecasting* 18.3 (2002), pp. 439–454. DOI: 10.1016/S0169-2070(01)00110-8.

[26] Ying Jin et al. *Bayesian Symbolic Regression*. 2019. arXiv: 1910.08892. URL: https://arxiv.org/abs/1910.08892.

[27] Pierre-Alexandre Kamienny et al. "End-to-end Symbolic Regression with Transformers". In: *Advances in Neural Information Processing Systems 35*. 2022. arXiv: 2204.10532. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/42eb37cdbefd7abae0835f4b67548c39-Abstract-Conference.html.

[28] Robert E. Kass and Adrian E. Raftery. "Bayes Factors". In: *Journal of the American Statistical Association* 90.430 (1995), pp. 773–795. DOI: 10.1080/01621459.1995.10476572.

[29] Guolin Ke et al. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree". In: *Advances in Neural Information Processing Systems 30*. 2017, pp. 3146–3154. URL: https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

[30] Hyunjik Kim and Yee Whye Teh. "Scaling up the Automatic Statistician: Scalable Structure Discovery using Gaussian Processes". In: *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*. Vol. 84. Proceedings of Machine Learning Research. PMLR, 2018, pp. 575–584. arXiv: 1706.02524. URL: https://proceedings.mlr.press/v84/kim18a.html.

[31] Urs Köster. *AutoBNN: Probabilistic time series forecasting with compositional Bayesian neural networks*. Google Research blog. Code in TensorFlow Probability, spinoffs/autobnn. Mar. 2024. URL: https://research.google/blog/autobnn-probabilistic-time-series-forecasting-with-compositional-bayesian-neural-networks/.

[32] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Complex Adaptive Systems. Cambridge, MA: The MIT Press, 1992. ISBN: 0-262-11170-5.

[33] William La Cava et al. "Contemporary Symbolic Regression Methods and their Relative Performance". In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. Vol. 1. 2021. arXiv: 2107.14351. URL: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c0c7c76d30bd3dcaefc96f40275bdc0a-Abstract-round1.html.

[34] Jung-Mok Lee et al. *Automated Model Discovery via Multi-modal & Multi-step Pipeline*. arXiv preprint; author names as listed on arXiv (Lee Jung-Mok, Nam Hyeon-Woo, Moon Ye-Bin). 2025. arXiv: 2509.25946. URL: https://arxiv.org/abs/2509.25946.

[35] Michael Y. Li, Emily B. Fox, and Noah D. Goodman. "Automated Statistical Model Discovery with Language Models". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 27791–27807. arXiv: 2402.17879. URL: https://proceedings.mlr.press/v235/li24v.html.

[36] James Robert Lloyd and Zoubin Ghahramani. "Statistical Model Criticism using Kernel Two Sample Tests". In: *Advances in Neural Information Processing Systems 28*. 2015, pp. 829–837. URL: https://proceedings.neurips.cc/paper/2015/hash/0fcbc61acd0479dc77e3cccc0f5ffca7-Abstract.html.

[37] James Robert Lloyd et al. "Automatic Construction and Natural-Language Description of Nonparametric Regression Models". In: *Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence*. Vol. 28. 1. AAAI Press, 2014. DOI: 10.1609/aaai.v28i1.8904. arXiv: 1402.4304. URL: https://ojs.aaai.org/index.php/AAAI/article/view/8904.

[38] Gustavo Malkomes, Charles Schaff, and Roman Garnett. "Bayesian Optimization for Automated Model Selection". In: *Advances in Neural Information Processing Systems 29*. 2016, pp. 2900–2908. URL: https://proceedings.neurips.cc/paper/2016/hash/3bbfdde8842a5c44a0323518eec97cbe-Abstract.html.

[39] Mistral AI. *Timestral 1.0 209M*. Model documentation; weights mistralai/Timestral-209M on Hugging Face (published 14 October 2026), modified MIT licence; no technical report found. 2026. URL: https://docs.mistral.ai/models/timestral-1-0-209m.

[40] Boris N. Oreshkin et al. "N-BEATS: Neural basis expansion analysis for interpretable time series forecasting". In: *International Conference on Learning Representations (ICLR)*. 2020. arXiv: 1905.10437. URL: https://iclr.cc/virtual_2020/poster_r1ecqn4YwB.html.

[41] Brenden K. Petersen et al. "Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients". In: *International Conference on Learning Representations (ICLR)*. 2021. arXiv: 1912.04871. URL: https://iclr.cc/virtual/2021/poster/2578.

[42] Jingang Qu et al. *NVIDIA Kumo Tabular Sets a New Accuracy-Efficiency Frontier for Tabular Prediction*. Hugging Face blog. Weights nvidia/Kumo-Tabular (OpenMDW 1.1); code https://github.com/NVIDIA/structured-data-models; no technical report found. Sept. 2026. URL: https://huggingface.co/blog/nvidia/kumo-tabular.

[43] Carl Edward Rasmussen and Hannes Nickisch. "Gaussian Processes for Machine Learning (GPML) Toolbox". In: *Journal of Machine Learning Research* 11.100 (2010), pp. 3011–3015. URL: https://jmlr.org/papers/v11/rasmussen10a.html.

[44] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. Cambridge, MA: The MIT Press, 2006. ISBN: 0-262-18253-X. URL: https://gaussianprocess.org/gpml/.

[45] Feras A. Saad. *AutoGP.jl: Automated Bayesian model discovery for time series data*. Software, Julia, Apache 2.0 licence. 2023. URL: https://github.com/probsys/AutoGP.jl.

[46] Feras A. Saad et al. "Bayesian Synthesis of Probabilistic Programs for Automatic Data Modeling". In: *Proceedings of the ACM on Programming Languages* 3.POPL (2019), 37:1–37:32. DOI: 10.1145/3290350. arXiv: 1907.06249.

[47] Feras A. Saad et al. "Sequential Monte Carlo Learning for Time Series Structure Discovery". In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 29473–29489. arXiv: 2307.09607. URL: https://proceedings.mlr.press/v202/saad23a.html.

[48] Michael Schmidt and Hod Lipson. "Distilling Free-Form Natural Laws from Experimental Data". In: *Science* 324.5923 (2009), pp. 81–85. DOI: 10.1126/science.1165893.

[49] Gideon Schwarz. "Estimating the Dimension of a Model". In: *The Annals of Statistics* 6.2 (1978), pp. 461–464. DOI: 10.1214/aos/1176344136.

[50] Parshin Shojaee et al. "LLM-SR: Scientific Equation Discovery via Programming with Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2025. arXiv: 2404.18400. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/28df8e730c054c5331855fd4d5403ba9-Abstract-Conference.html.

[51] Parshin Shojaee et al. "LLM-SRBench: A New Benchmark for Scientific Equation Discovery with Large Language Models". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 55325–55359. arXiv: 2504.10415. URL: https://proceedings.mlr.press/v267/shojaee25a.html.

[52] Christian Steinruecken et al. "The Automatic Statistician". In: *Automated Machine Learning: Methods, Systems, Challenges*. Ed. by Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. The Springer Series on Challenges in Machine Learning. Springer, 2019, pp. 161–173. DOI: 10.1007/978-3-030-05318-5_9.

[53] Trevor Stephens. *gplearn: Genetic Programming in Python, with a scikit-learn inspired API*. Software, BSD 3-clause licence; repository created March 2015. 2015. URL: https://github.com/trevorstephens/gplearn.

[54] Shengyang Sun et al. "Differentiable Compositional Kernel Learning for Gaussian Processes". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 4828–4837. arXiv: 1806.04326. URL: https://proceedings.mlr.press/v80/sun18e.html.

[55] Richard Cornelius Suwandi et al. "Adaptive Kernel Design for Bayesian Optimization Is a Piece of CAKE with LLMs". In: *Advances in Neural Information Processing Systems 38*. 2025. arXiv: 2509.17998. URL: https://neurips.cc/virtual/2025/poster/119378.

[56] Silviu-Marian Udrescu and Max Tegmark. "AI Feynman: A physics-inspired method for symbolic regression". In: *Science Advances* 6.16 (2020), eaay2631. DOI: 10.1126/sciadv.aay2631. arXiv: 1905.11481.

[57] Silviu-Marian Udrescu et al. "AI Feynman 2.0: Pareto-optimal symbolic regression exploiting graph modularity". In: *Advances in Neural Information Processing Systems 33*. 2020. arXiv: 2006.10782. URL: https://proceedings.neurips.cc/paper/2020/hash/33a854e247155d590883b93bca53848a-Abstract.html.

[58] Marco Virgolin and Solon P. Pissis. "Symbolic Regression is NP-hard". In: *Transactions on Machine Learning Research* (2022). arXiv: 2207.01018. URL: https://arxiv.org/abs/2207.01018.

[59] Andrew Gordon Wilson and Ryan Prescott Adams. "Gaussian Process Kernels for Pattern Discovery and Extrapolation". In: *Proceedings of the 30th International Conference on Machine Learning*. Vol. 28. Proceedings of Machine Learning Research 3. PMLR, 2013, pp. 1067–1075. arXiv: 1302.4245. URL: https://proceedings.mlr.press/v28/wilson13.html.

[60] Gerald Woo et al. "Unified Training of Universal Time Series Forecasting Transformers". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 53140–53164. arXiv: 2402.02592. URL: https://proceedings.mlr.press/v235/woo24a.html.